



Course Code: MCT444s

Mechatronics in Rehabilitation Technology

Lower Limb Exoskeleton for Knee Rehabilitation

OpenSim-Based Simulation and PID Control

Team Members

Name	Student ID
Abdullah Mahmoud Abdelkarim	2100658
Abdelkarim Wael Abdelkarim	2101733
Mahmoud Salama Mohamed Abdo	2100961
Moaz Gamal ALSayed	2101231
Musa Gamal El-Shenawy	2100614

[Academic Year — 2025/2026, Semester 2]

1. Table of Contents

1. Table of Contents	2
2. Table Of Figures	2
3. Introduction and Project Motivation	3
4. System Overview and Models	4
4.1. CAD Model of the Exoskeleton	4
4.2. Human Body Musculoskeletal Model	5
4.3. Exoskeleton Placement on the Human Body	5
5. OpenSim Simulation Pipeline	6
5.1. Inverse Kinematics (IK)	6
5.2. Inverse Dynamics (ID)	6
5.3. Residual Reduction Algorithm (RRA)	7
5.4. Computed Muscle Control (CMC)	7
5.5. Static Optimization	8
5.6. Forward Dynamics (FWD)	8
5.7. Segment Tree Topology View	9
6. MATLAB PID Controller	10
6.1. Controller Flowcharts	11
7. Metabolic Cost Analysis	12
8. Simulation Trials and Results	13
8.1. Trial Overview	13
8.2. Muscle Activation Results	13
8.3. Muscle Force Results	15
8.4. Quantitative Results Tables	16
8.5. Summary of Energy Reduction	17
9. Discussion	18
10. Conclusion	19
11. References+Video_link	20

2. Table Of Figures

Figure 1 .Initial CAD model of the exoskeleton prior to geometry adjustments	4
Figure 2 .Modified CAD model tailored for musculoskeletal integration	4
Figure 3 .Exoskeleton placed and aligned on the right knee of the musculoskeletal model	5
Figure 4 Inverse Kinematics setup and execution in OpenSim	6
Figure 5 .Inverse Dynamics tool calculating net joint moments	6
Figure 6 .Residual Reduction Algorithm adjusting model dynamics	7
Figure 7 .Computed Muscle Control generating baseline muscle excitations	7
Figure 8 Static Optimization process for muscle force	8
Figure 9 .Forward Dynamics simulation verifying targeted motion	8
Figure 10 .Hierarchical segment tree topology confirming actuator attachments	9
Figure 11 High-level flowchart of the MATLAB simulation and control loop	11
Figure 12 Block diagram illustrating the closed-loop PD control architecture.	11
Figure 13 Metabolic probe integration within the OpenSim environment.	12
Figure 14 Exoskeleton assistive torque output during the successful trial.	13
Figure 15 Comparison of Sartorius muscle activity (Base vs. Assisted).	13
Figure 16 Comparison of Semimembranosus muscle activity (Base vs. Assisted).	14
Figure 17 Comparison of Biceps Femoris Short Head muscle activity (Base vs. Assisted).	14
Figure 18 Muscle force generation comparison for the Sartorius muscle.	15
Figure 19 Muscle force generation comparison for the Biceps Femoris Short Head.	15
Figure 20 Comparison of base area vs. active area for muscle activation.	16
Figure 21 Comparison of base area vs. active area for muscle activation.	16
Figure 22 Metabolic energy expenditure reductions per analyzed muscle.	17

3. Introduction and Project Motivation

Knee rehabilitation is an essential part of recovery for patients suffering from stroke, ACL injuries, neurological disorders, or lower-limb musculoskeletal impairments. These patients often experience reduced mobility, abnormal gait patterns, muscle weakness, and reduced joint stability. Conventional rehabilitation procedures depend heavily on repetitive therapist-assisted exercises, which can be physically exhausting and difficult to standardize.

Wearable robotic exoskeletons provide an alternative rehabilitation approach by generating controlled assistive torques around the joints of the lower limb. In knee rehabilitation, an exoskeleton can support flexion and extension movements during walking, reducing muscular effort while preserving physiologically meaningful gait motion.

The objective of this project is to develop a complete OpenSim-based simulation framework for a lower-limb knee exoskeleton. The project integrates a musculoskeletal gait model, a CAD-based exoskeleton, and a MATLAB PID controller. The exoskeleton actuator applies assistive torque to the right knee joint, and the effects of assistance are evaluated through muscle activation analysis, muscle force analysis, and metabolic energy estimation.

The primary software tools used in the project are:

- OpenSim for musculoskeletal modeling and biomechanical simulation.
- MATLAB for controller implementation and closed-loop simulation.
- CAD software for exoskeleton geometry editing and alignment.

4. System Overview and Models

4.1. CAD Model of the Exoskeleton

The exoskeleton CAD model used in this project was obtained from an open-source online repository.

Repository source: [Link to CAD model repository](#)

The imported CAD model was modified to better match the requirements of the simulation and the geometry of the musculoskeletal model. The modifications included adjustments to component geometry, actuator mount positioning, and alignment around the knee joint.

These modifications were performed to optimize joint alignment and improve actuator attachment positioning.

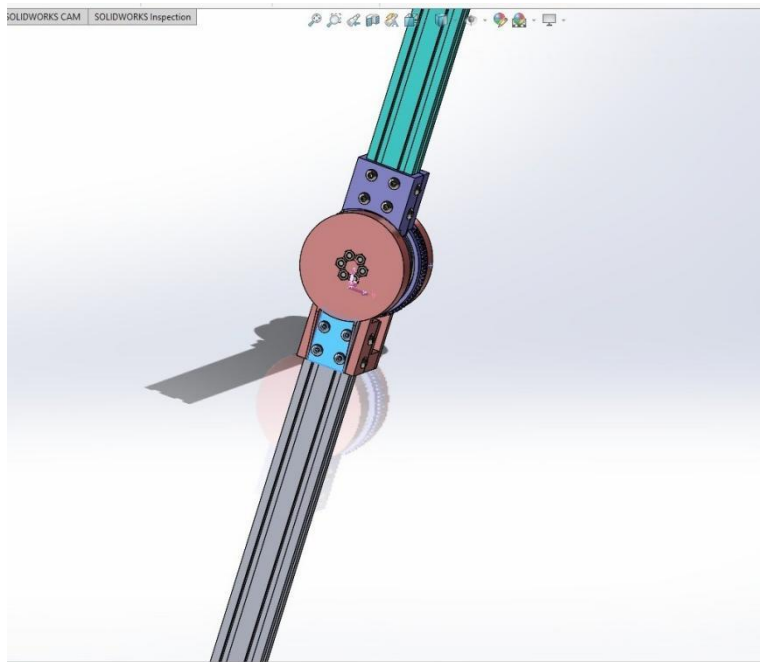


Figure 1. Initial CAD model of the exoskeleton prior to geometry adjustments

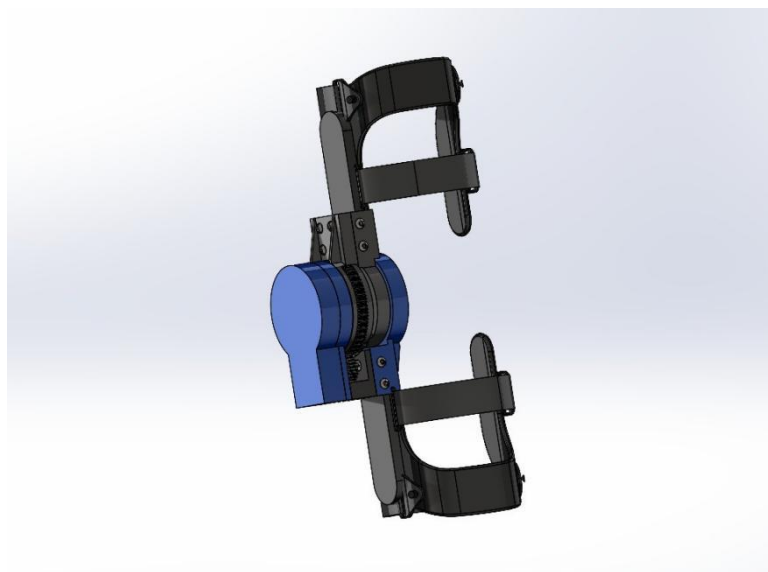


Figure 2. Modified CAD model tailored for musculoskeletal integration

4.2. Human Body Musculoskeletal Model

The musculoskeletal model used in this project is the OpenSim Gait2392 Simbody model.

Repository source: [Link to musculoskeletal model repository](#)

The Gait2392 model includes 23 degrees of freedom and 92 muscle-tendon actuators. It is widely used for gait analysis and lower-limb biomechanics research.

The specific model file used in the project is: `subject01_simbody_adjusted.osim`

This file represents a subject-specific adjusted version of the original Gait2392 model.

4.3. Exoskeleton Placement on the Human Body

The modified exoskeleton model was attached to the musculoskeletal model inside OpenSim.

A `CoordinateActuator` named `exo_knee_r` was added to the right knee coordinate `knee_angle_r`. This actuator represents the torque-producing mechanism of the exoskeleton.

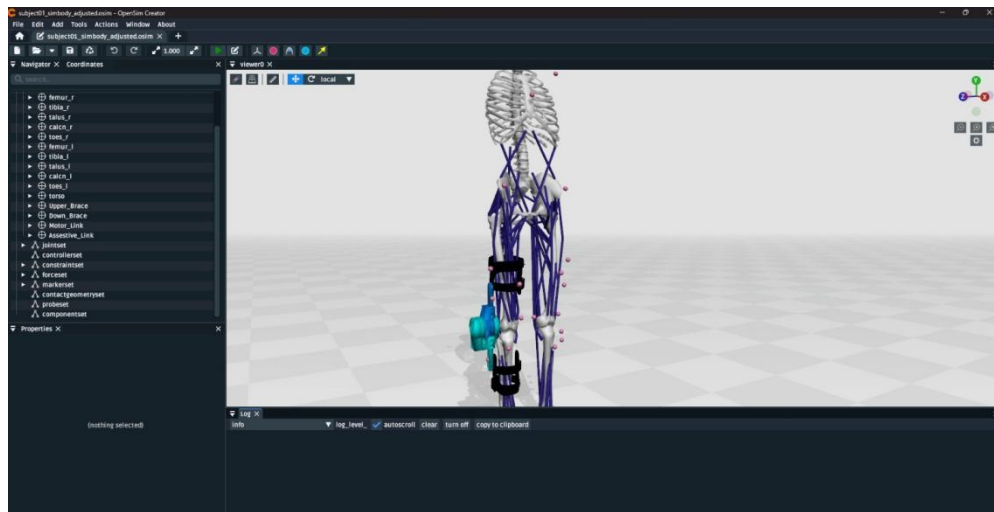


Figure 3. Exoskeleton placed and aligned on the right knee of the musculoskeletal model

5. OpenSim Simulation Pipeline

5.1. Inverse Kinematics (IK)

Inverse Kinematics is the first stage of the simulation pipeline. IK computes the generalized coordinates of the musculoskeletal model that best match experimentally measured marker trajectories during walking.

This step determines the joint angles and segment positions throughout the gait cycle. The output of IK is a motion file containing the time-varying joint coordinates.

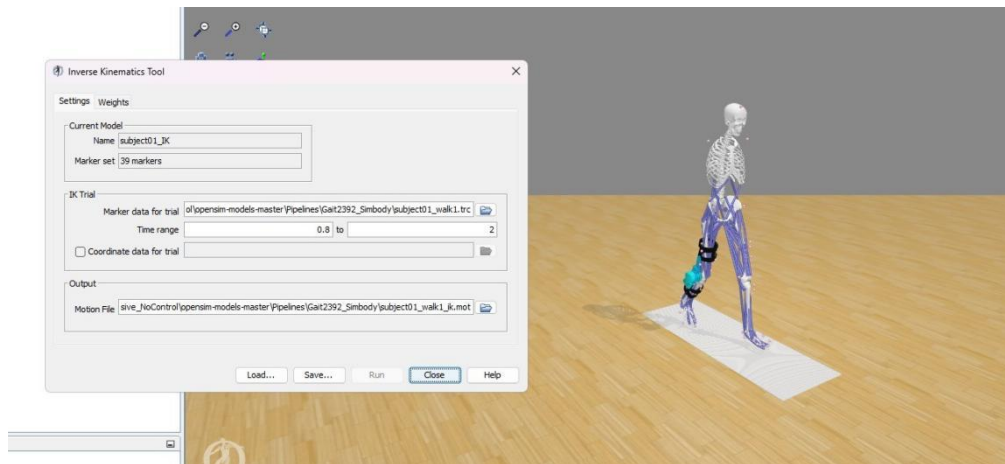


Figure 4. Inverse Kinematics setup and execution in OpenSim

5.2. Inverse Dynamics (ID)

Inverse Dynamics computes the net joint moments required to generate the observed motion using the kinematics from IK and the measured external forces.

The resulting joint torques provide information about the mechanical demands at the joints during gait.

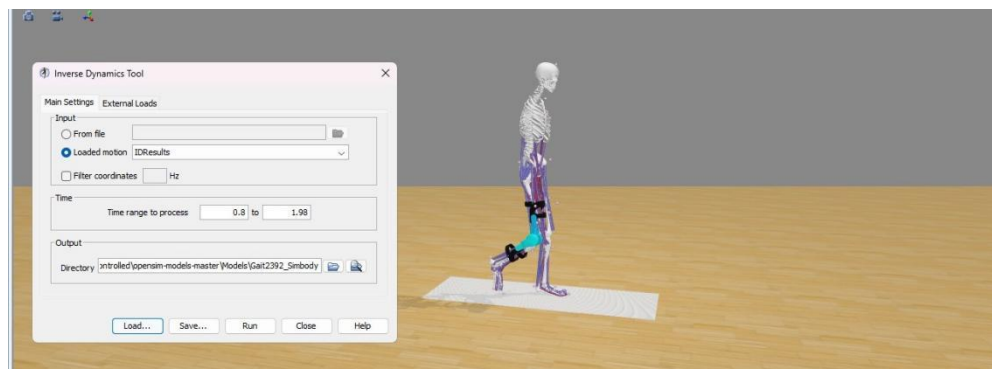


Figure 5. Inverse Dynamics tool calculating net joint moments

5.3. Residual Reduction Algorithm (RRA)

The Residual Reduction Algorithm reduces residual forces and moments caused by dynamic inconsistencies between kinematic and force data.

RRA improves simulation stability by slightly adjusting body segment masses and modifying kinematics to improve dynamic consistency.

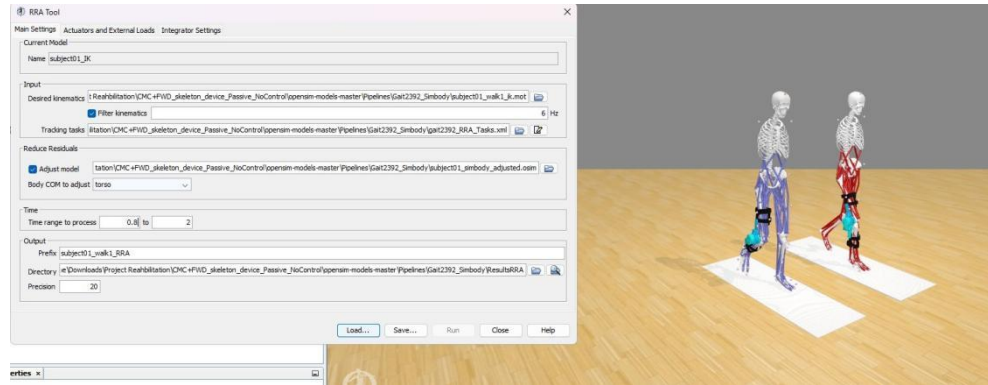


Figure 6. Residual Reduction Algorithm adjusting model dynamics

5.4. Computed Muscle Control (CMC)

Computed Muscle Control calculates the muscle excitation patterns required to reproduce the desired gait motion.

CMC produces:

- subject01_walk1_states.sto
- subject01_walk1_controls.sto

These outputs are later used as reference inputs for the MATLAB PID controller.

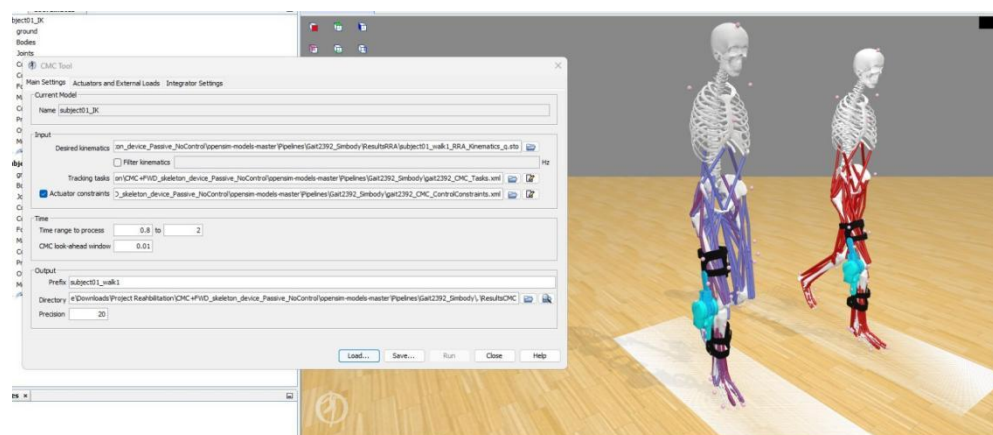


Figure 7. Computed Muscle Control generating baseline muscle excitations

5.5. Static Optimization

Static Optimization decomposes the joint moments into individual muscle activations and muscle forces.

The optimization process minimizes muscle effort while satisfying the required joint moments.

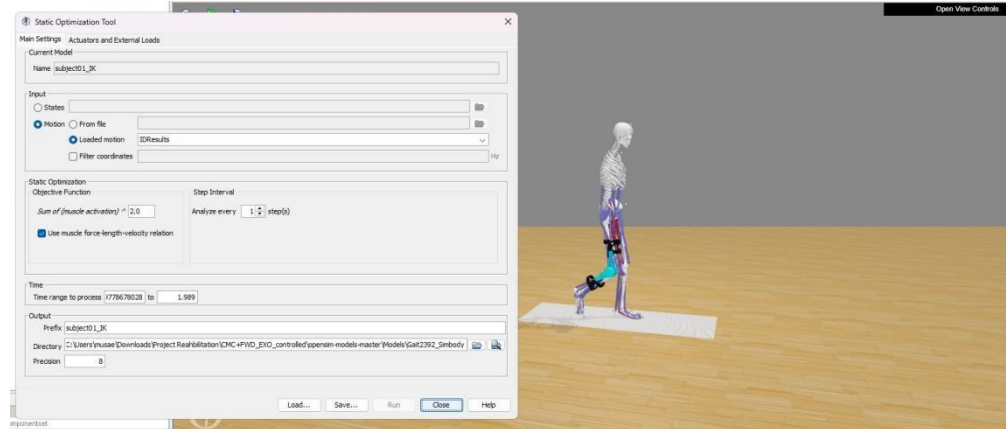


Figure 8 Static Optimization process for muscle force

5.6. Forward Dynamics (FWD)

Forward Dynamics simulates the motion of the musculoskeletal model forward in time using muscle controls and exoskeleton torques.

This stage verifies whether the generated controls successfully reproduce the target motion.

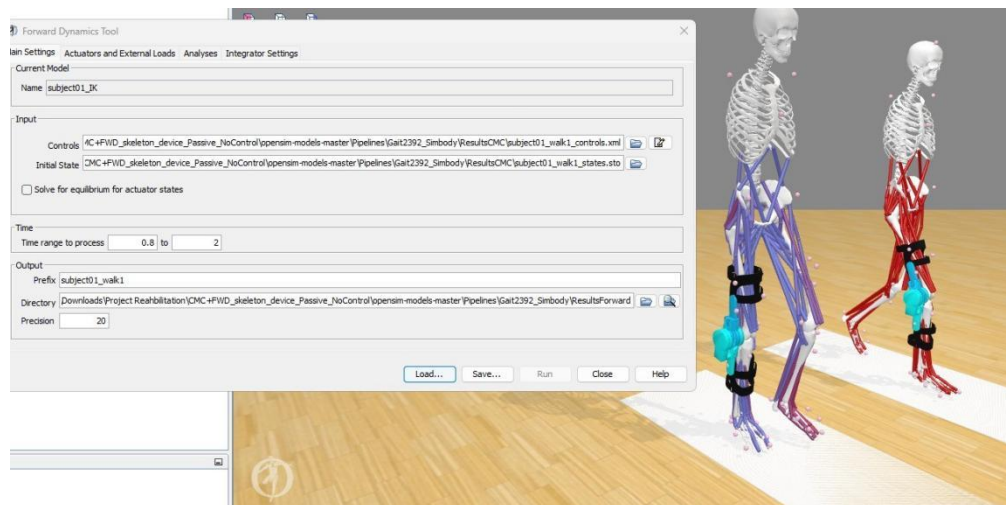


Figure 9. Forward Dynamics simulation verifying targeted motion

5.7. Segment Tree Topology View

The segment tree topology represents the hierarchical structure of the musculoskeletal model.

This view is useful for verifying that the actuator `exo_knee_r` is correctly attached to the right knee joint.

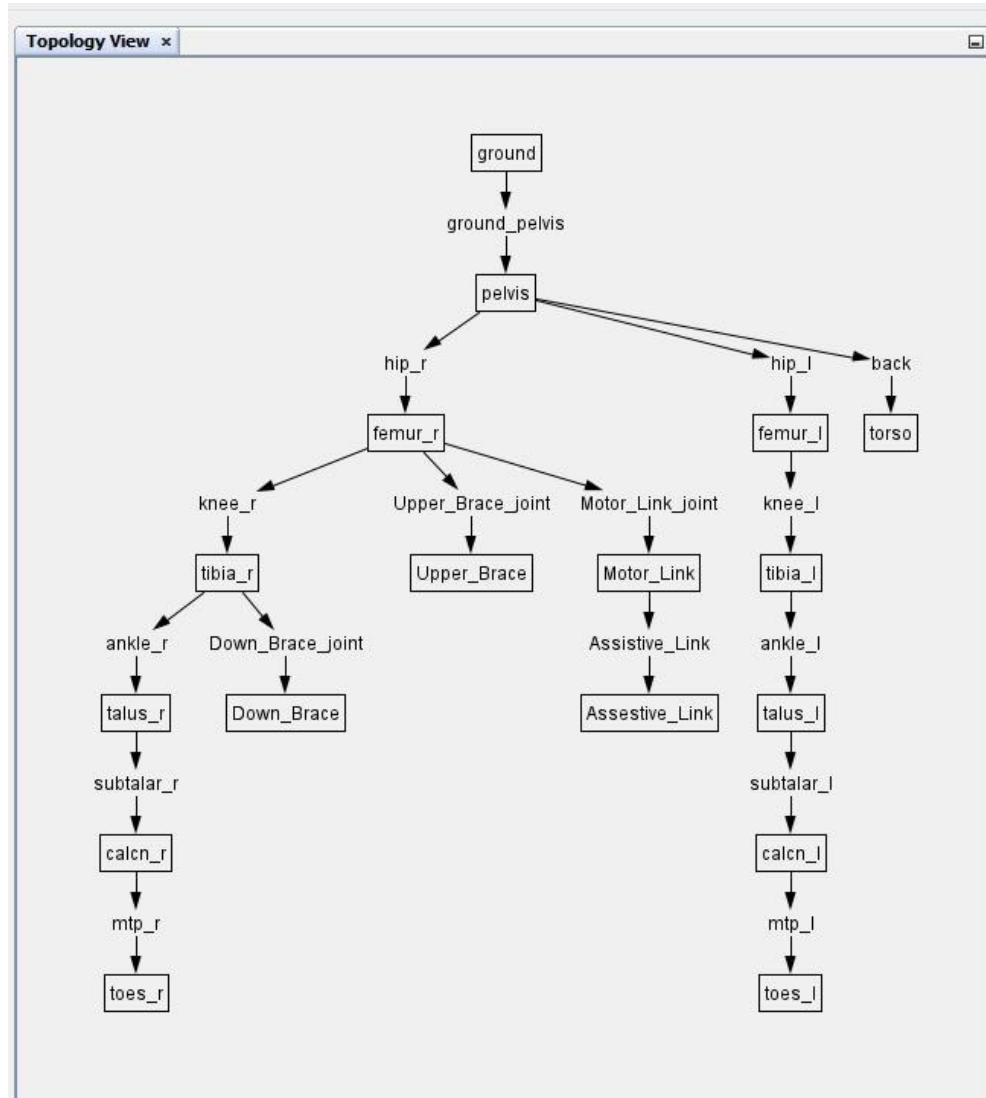


Figure 10. Hierarchical segment tree topology confirming actuator attachments

6. MATLAB PID Controller

Control Strategy Overview

The controller applies assistive torque to the right knee joint through the exoskeleton actuator `exo_knee_r`. The implementation bridges MATLAB and OpenSim via the OpenSim API, creating a closed-loop framework to actively assist the gait cycle.

Based on the implemented MATLAB script, the simulation execution and control logic are broken down as follows:

- **Initialization and Pre-loading:** The OpenSim model (`subject01_simbody_adjusted.osim`) is loaded alongside reference kinematic states and baseline CMC muscle controls. To optimize simulation speed and prevent execution delays during the main loop, these reference trajectories and muscle excitations are pre-loaded directly into fast-access MATLAB arrays.
- **Simulation Timeframe:** The simulation integrates forward from $t_0 = 0.53$ s to $t_f = 2.00$ s with a discrete time step of $dt = 0.01$ s.
- **Error Calculation:** At each time step, the script interpolates the target reference angle and angular velocity. It compares these desired states to the actual simulated model states to determine the real-time position error and velocity error.
- **Derivative Filtering:** To prevent erratic torque spikes caused by computational noise or rapid velocity changes, a low-pass filter ($\alpha = 0.85$) is applied to the derivative error signal.
- **Gain Ramping:** To ensure a smooth onset of exoskeleton assistance and maintain the musculoskeletal model's dynamic stability, the controller utilizes a gain ramp-in mechanism over the first 0.05 seconds. The effective control gains linearly increase from zero until they reach their maximum tuned values.
- **Control Law & Saturation:** The core control law computes the required torque using the effective Proportional ($K_p = 100$) and Derivative ($K_d = 20$) gains. While an Integral gain ($K_i = 5$) and an integral error accumulator are initialized, the primary corrective torque is driven by the PD components. The final torque command is strictly saturated to a limit of ± 35 Nm to emulate realistic, safe hardware limitations.
- **Actuation Override:** Both the baseline physiological muscle controls and the newly computed robotic exoskeleton torque are enforced simultaneously using OpenSim's `overrideActuation` command. The system is then passed to the Manager to integrate the state to the next time step.

6.1. Controller Flowcharts

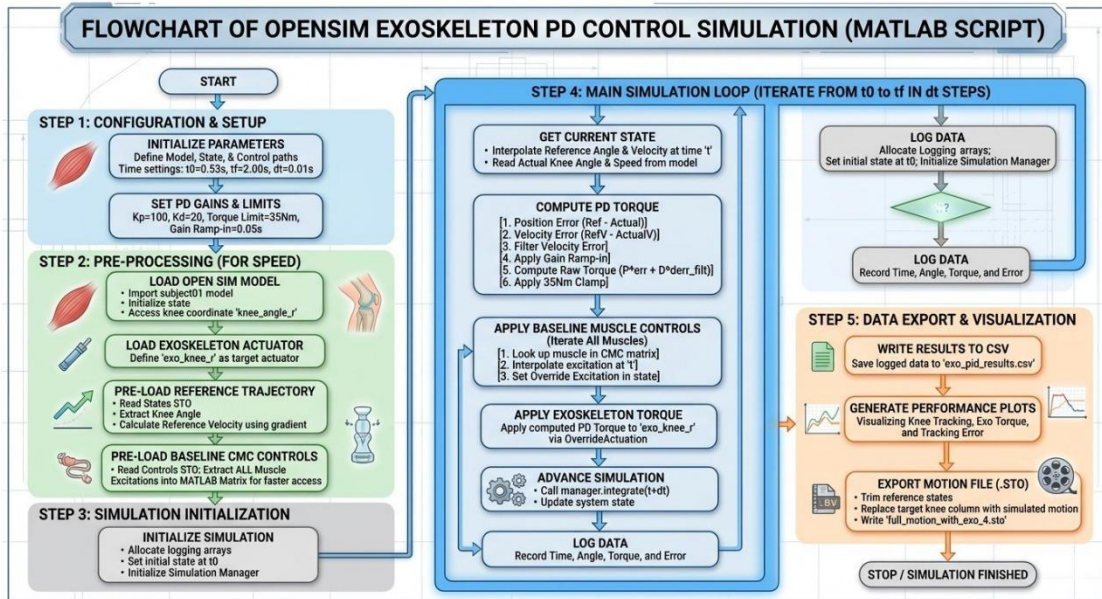


Figure 11 High-level flowchart of the MATLAB simulation and control loop.

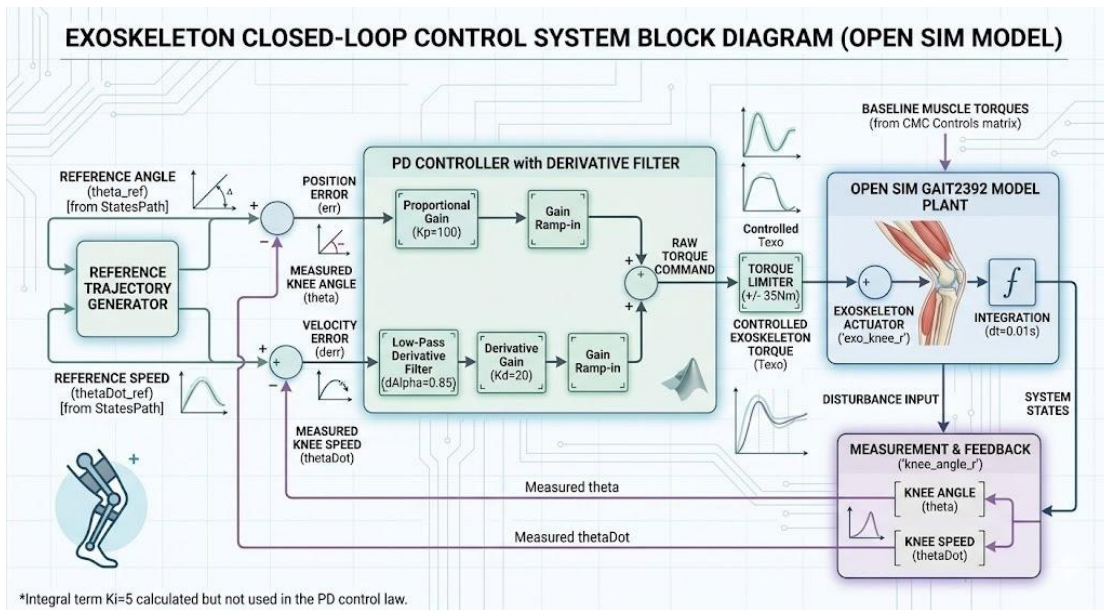


Figure 12 Block diagram illustrating the closed-loop PD control architecture.

7. Metabolic Cost Analysis

A metabolic probe was added to the OpenSim model to estimate metabolic energy consumption during simulation.

The analysis process included:

1. Running Static Optimization.
2. Computing muscle power and heat generation.
3. Integrating energy over time.

Filtering techniques were applied to reduce noise in the raw signals before energy integration.

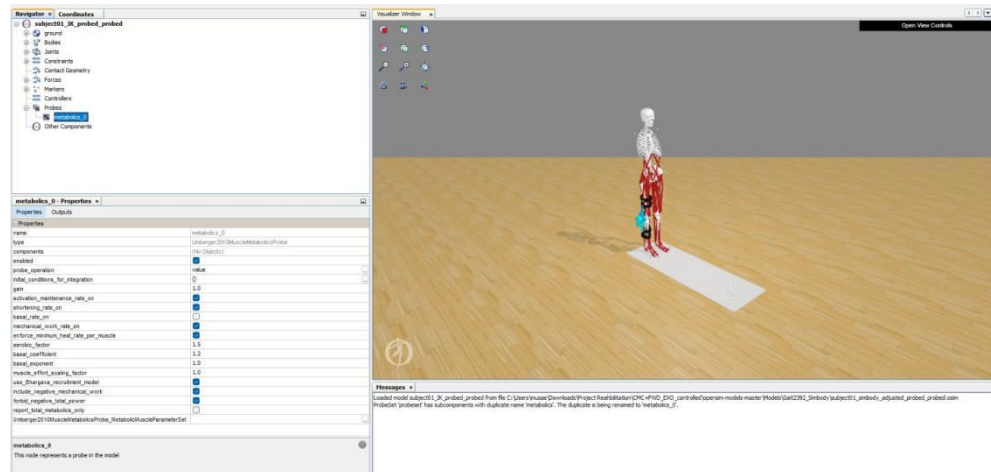


Figure 13 Metabolic probe integration within the OpenSim environment.

8. Simulation Trials and Results

8.1. Trial Overview

Three simulation trials were performed.

Trial 1 and Trial 2 were unsuccessful due to controller divergence or simulation instability. These failed trials were important for identifying instability sources and improving the simulation pipeline.

Trial 3 produced a stable and converged simulation.

8.2. Muscle Activation Results

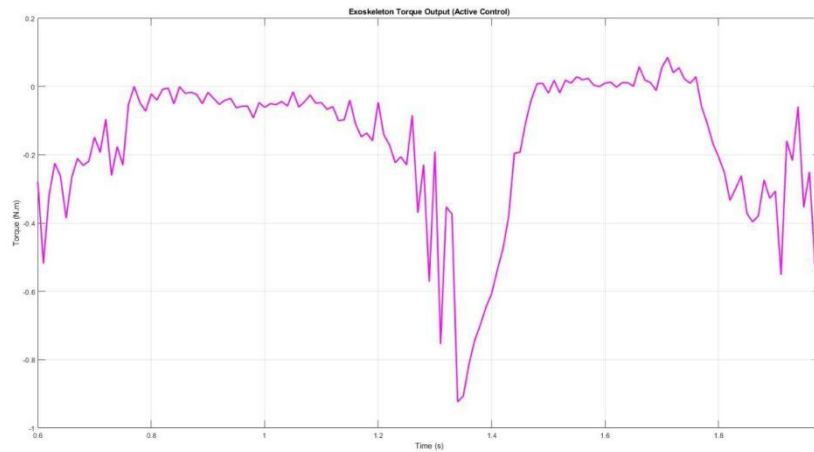


Figure 14 Exoskeleton assistive torque output during the successful trial.

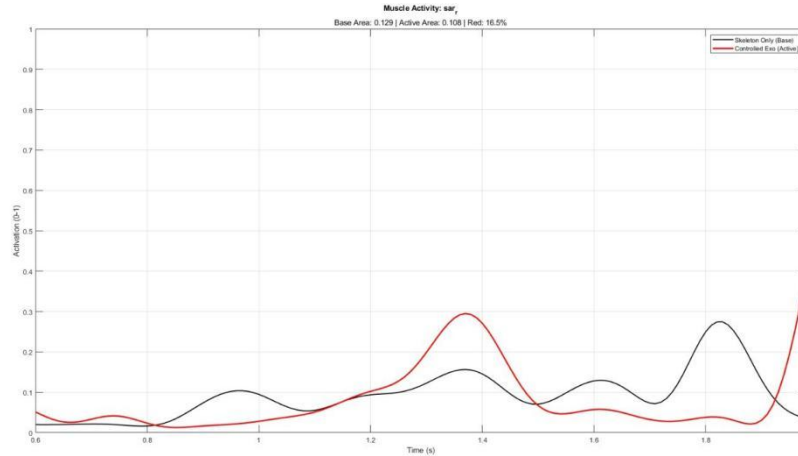


Figure 15 Comparison of Sartorius muscle activity (Base vs. Assisted).

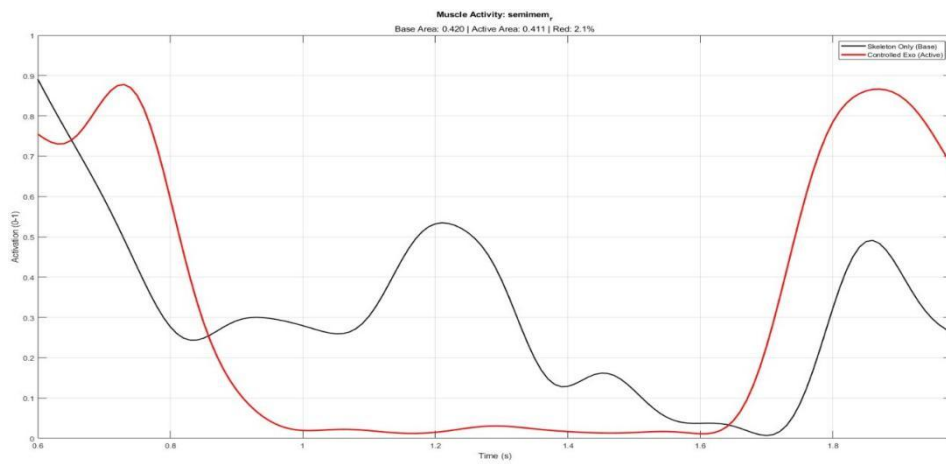


Figure 16 Comparison of Semimembranosus muscle activity (Base vs. Assisted).

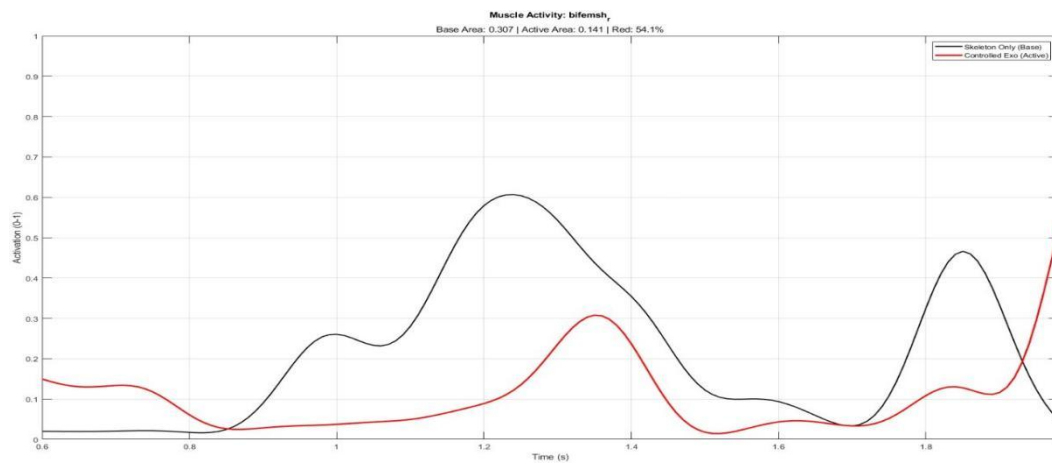


Figure 17 Comparison of Biceps Femoris Short Head muscle activity (Base vs. Assisted).

8.3. Muscle Force Results

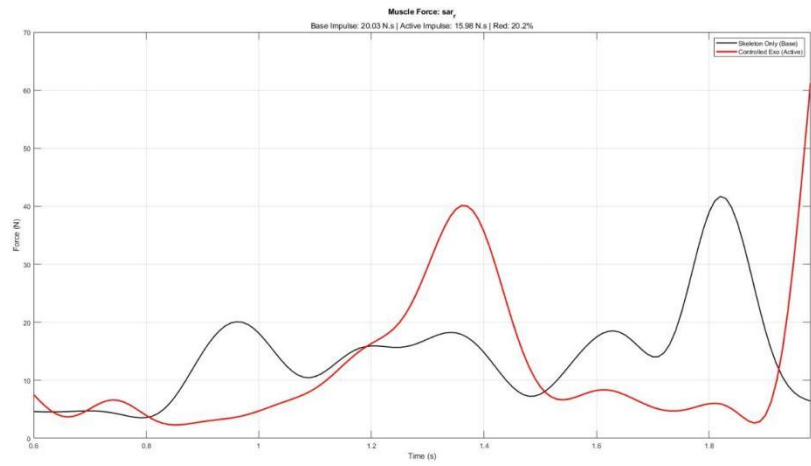


Figure 18 Muscle force generation comparison for the Sartorius muscle.

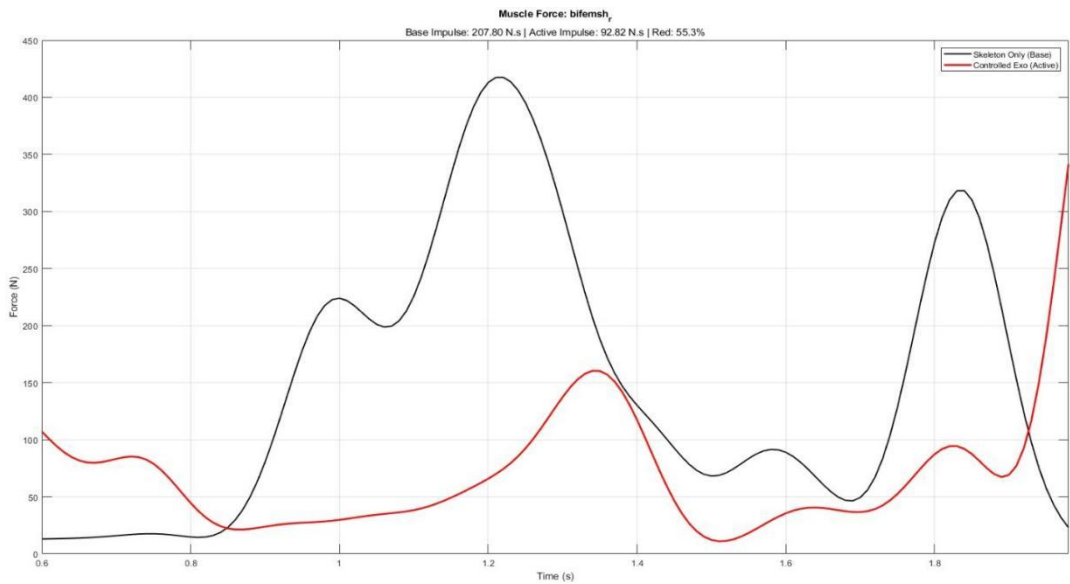


Figure 19 Muscle force generation comparison for the Biceps Femoris Short Head.

8.4. Quantitative Results Tables

Table 1: Muscle Force Impulse Analysis

```

--- Muscle Force Analysis: Base Impulse vs Active Impulse ---
Muscle           | Base Impulse | Active Impulse | Reduction %
-----
vas_med_r        | 132.35       | 45.43          | 65.7      %
vas_lat_r        | 288.00       | 58.97          | 79.5      %
vas_int_r        | 157.15       | 47.41          | 69.8      %
bifemsh_r        | 207.80       | 92.82          | 55.3      %
sar_r            | 20.03        | 15.98          | 20.2      %
fx >>

```

Figure 20 Comparison of base area vs. active area for muscle activation.

Table 2: Muscle Activation Analysis

```

--- Muscle Activation Analysis: Base Area vs Active Area ---
Muscle           | Base Area    | Active Area    | Reduction %
-----
vas_med_r        | 0.1609       | 0.0423         | 73.7      %
vas_lat_r        | 0.1462       | 0.0391         | 73.2      %
vas_int_r        | 0.1574       | 0.0415         | 73.6      %
bifemsh_r        | 0.3072       | 0.1411         | 54.1      %
semimem_r        | 0.4199       | 0.4113         | 2.1       %
sar_r            | 0.1294       | 0.1080         | 16.5      %

```

Figure 21 Comparison of base area vs. active area for muscle activation.

Table 3: Metabolic Energy Consumption per Muscle

=====				
ANALYSIS RESULTS: METABOLIC ENERGY CONSUMPTION				
=====				
Total Energy (Base Case): 90.3064 J				
Total Energy (Assist Case): 85.1599 J				
Overall Reduction: 5.70%				

Muscle	Base_Energy_J	Assist_Energy_J	Difference	Reduction_Percent

{'rect_fem_r'}	0.69607	0.51439	0.18167	26.1
{'vas_med_r' }	3.157	2.9593	0.19777	6.2645
{'vas_lat_r' }	4.5481	4.2953	0.25278	5.5581
{'vas_int_r' }	3.2663	3.0619	0.20446	6.2597
{'bifemlh_r' }	1.5183	1.5067	0.011591	0.76342
{'semimem_r' }	3.0268	2.8608	0.166	5.4843

Figure 22 Metabolic energy expenditure reductions per analyzed muscle.

8.5. Summary of Energy Reduction

The base case total metabolic energy was 90.3064 J, while the assisted case total metabolic energy was 85.16 J, yielding an overall reduction of 5.7%.

9. Discussion

The PID controller successfully tracked the reference knee angle trajectory with acceptable stability.

Reductions in activation and force within the vastus muscle group indicate that the exoskeleton effectively unloaded the quadriceps muscles.

Changes in hamstring muscle behavior may indicate compensatory stabilization effects during assisted gait.

The reduction in metabolic energy demonstrates that the exoskeleton reduced muscular demand during walking.

Several limitations remain:

- the model is generic,
- ground reaction forces are idealized,
- PID gains were manually tuned.

Future work may explore adaptive control or model predictive control approaches.

10. Conclusion

This project successfully developed a full OpenSim simulation pipeline for a knee rehabilitation exoskeleton.

A MATLAB PID controller was implemented to generate assistive torque during gait simulation.

The assisted case demonstrated reductions in:

- muscle activation,
- muscle force,
- metabolic energy consumption.

The project demonstrates the feasibility of model-based exoskeleton control for rehabilitation engineering applications.

11. References

1. <https://github.com/Malaphor/exo-legs/tree/master>
2. <https://github.com/opensim-org/opensim-models>
3. Delp, S.L., et al. (2007). OpenSim: Open-source software to create and analyze dynamic simulations of movement.
4. Thelen, D.G., Anderson, F.C., & Delp, S.L. (2003). Generating dynamic simulations of movement using computed muscle control.

12. Video Link